

Business-Driven Product Planning Using Feature Vectors and Increments

Brian A. Nejme, *Instep*

Ian Thomas, *Ian Thomas and Associates*

Software product development organizations are frequently hampered by all-or-nothing approaches to product planning driven by technical issues. We present a method for business-driven product planning based on the notions of feature vectors, feature levels, and release increments. This method provides a decision framework within which to assess the range of possible combinations of features, times, and costs and to define release schedules with desirable content, predictable

costs, and firm schedules. This incremental product-planning method provides significant product release flexibility crucial to both start-up and mature software companies.

To illustrate how to use our framework, we present a case study of a company developing software and hardware Internet access equipment. The specific features and values we cite are representative, but, to protect proprietary information, they are not the actual values.

This article focuses principally on software product companies engaged in developing software products for commercial sale. We believe our incremental product-planning method would prove a critical success factor for any software product organization, especially start-ups determining initial and evolving product release functionality. More mature organizations have found the method useful in planning the evolution of more established products. However, our

framework also has relevance for software professionals who work in corporate IT environments and are planning a series of releases of their information systems, software companies using an application service provider (ASP) model, and systems companies whose systems contain a significant software component. In general, our product-planning method is relevant to anyone involved with selecting a balance of product features to maximize business value to the development organization.

Commercial software product-planning challenges

From the perspective of software product planning, we live in times of unprecedented haste. In the mid-1990s, commercial software enterprises typically measured product release cycles in one- to two-year increments. Today, many factors are driving

Agile development processes emphasize prioritizing product features, but most don't tell you how to do it. This product-development framework offers a method of defining release increments and assessing product features to deliver the highest return on development investment.

product release cycles to months instead of years. Competition, for example, has become a critical force in product planning. Competition takes many forms, including

- *Time to market.* A study of the technology industry found that products that come to market six months late but within budget earn 33 percent less profit over a five-year period compared to products out on time.¹ The study also found that bringing a product to market on time but 50 percent over budget cuts profits by only 4 percent.
- *Agile new entrants.* Powerful software development frameworks such as J2EE (www.java.sun.com/j2ee) and .NET (www.microsoft.com/net) and the open source movement² have facilitated rapid development of competitive products, giving birth to formidable competitors in most software markets. In addition, the low-cost, broad deployment, and product feedback options that the Internet affords enable unprecedented levels of timely product deployment, feedback, and revision.

Many product release plans depend on a single definition of the target release's contents. This makes it difficult to reduce the product scope to meet delivery schedules, to respond to changes in market conditions, or to launch new products in the case of a start-up. Rather than defining incrementally releasable products, most product plans tend to focus on monolithic, large-grained releases. However, more than ever, today's product planners must preserve flexibility as they define products—both in the product's functionality and in the release calendar. In addition, the heightened competition means that companies must maximize the value of their development effort for new releases.

The goal of software product planning should be to maximize the product's value within available resources—that is, to deliver the highest return on development investment, or ROI.

Meeting the challenges

These challenges have generated several responses, with *evolutionary* and *agile* development methods playing leading roles (see www.gilb.com).³ An agile process is a

development approach that is adaptive rather than predictive in nature, and people-oriented rather than process-oriented.³ Extreme Programming is one well-known example. This article complements some of the fundamental premises of agile process methods, including

- Incremental and highly iterative development
- Frequent product releases
- Development priority and sequencing driven by business value, as expressed by business stakeholders

Both agile and evolutionary process methods emphasize close contact between the development team and stakeholders. However, these methods assume that stakeholders can communicate business requirements effectively and reorient development as increments are released. In addition, these methods provide no guidance on several key issues:

- They don't address how the development group should interact with the key stakeholders and the product manager, who determines the product definition in most companies.
- They don't describe how the product manager and the development group can produce consistent business feature requirements and priorities that will satisfy multiple customers.
- They don't suggest techniques for the product manager to balance the often conflicting demands of multiple customers.
- They don't provide techniques for product managers to assess the value of proposed features.

In some of the agile process approaches, effective requirements prioritization is implicit. Tom Gilb and Karl Wieggers have addressed this issue explicitly (see www.gilb.com).^{3,4} We extend these approaches using the concept of explicit functional levels for each product feature. We extend the established notion of a release increment by relating the increment's definition explicitly to the product characteristics a product manager is charged with managing. Our approach also emphasizes that the product manager performs product planning to help the company meet its objectives—a goal

The goal of software product planning should be to maximize the product's value within available resources.

Software product organizations often have difficulty making decisions that transcend the purely technical, especially when it comes to product planning.

that might be different from maximizing a release's value to customers.

Effective product planning cannot separate the product manager's planning processes and techniques from the development team's development approaches. The product-planning framework we propose integrates well with incremental development methods and is a principal contribution of our work. Our approach builds on some of the models used at Microsoft.⁵

Product-planning and development processes

Solutions for effective software development inhabit the broader context of systematic product planning. However, software product organizations often have difficulty making decisions that transcend the purely technical, especially when it comes to product planning. Nevertheless, thinking beyond purely technical issues is critical to these organizations' business success. Thus, besides integrating with evolutionary and agile development techniques, our product-planning framework incorporates business and economic drivers such as time to market and ROI for engineering effort. It also emphasizes a cross-functional process that includes senior management, product managers, software engineers, and other key business functions.

The product manager

In a commercial-product setting, a product manager coordinates the activities associated with developing and commercializing a product or product line. Product manager responsibilities include product planning, partnerships, pricing, product rollout, and so on. In larger companies, the product manager often chairs a product-planning working group consisting of domain experts and representatives from sales, marketing, development, services, and support. In many companies, the product manager is responsible for the product line's overall well-being and vitality, including profitability.

For product planning, the product manager heads the process of considering all inputs and producing a release increment plan. The company's business plan, strategy, and budget determine the process's constraints. Inputs to product planning include data and perspectives from many stakeholders: prospects, customers, win-and-loss reports, problem re-

ports, enhancement requests, sales and field feedback, competitive and market analysis, new technologies, development, and customer support. The principal output from the planning process is a rolling release plan that identifies a sequence of release increments with defined functionality and target dates. Often, trying to satisfy all the stakeholders with the contents of a release leads to plans that are too complex, too costly, and too late. Our approach lets the product manager obtain input from the stakeholders so that he or she can make decisions to define release increments that provide real value and can be delivered on time and within budget.

Incremental product planning: Key concepts

A *release objective* is a high-level statement of a major goal for a product release. Typically, each release has two release objectives. The release objectives drive overall release contents and act as a filter for candidate release features. At least one release objective should have obvious relevance to the end-user community. Release objectives might relate to functional areas, usability improvements, support for third-party products and standards, workflow improvements, performance enhancements, product stabilization, and so on. In our case study, the principal objectives for the initial release were to achieve data transmission performance comparable with existing and competitive products and a consumer cost no greater than existing or competitive products.

A *feature category* provides a grouping mechanism for related functional capabilities. A *feature* is an individual functional capability that belongs to a feature category. Effective product planning requires identifying possible features over which planning decisions are made. We begin by breaking the product's behavior into feature categories. In our case study there were seven categories—five hardware, two software. The seven categories included a total of 35 features, 23 of which were primarily software related. For one of the software feature categories—software performance—we identified protocol implementation and bandwidth management as two features. For the software usability feature category, we identified software upgradability features for different components, a bandwidth management feature, and operations tools that managed the opera-

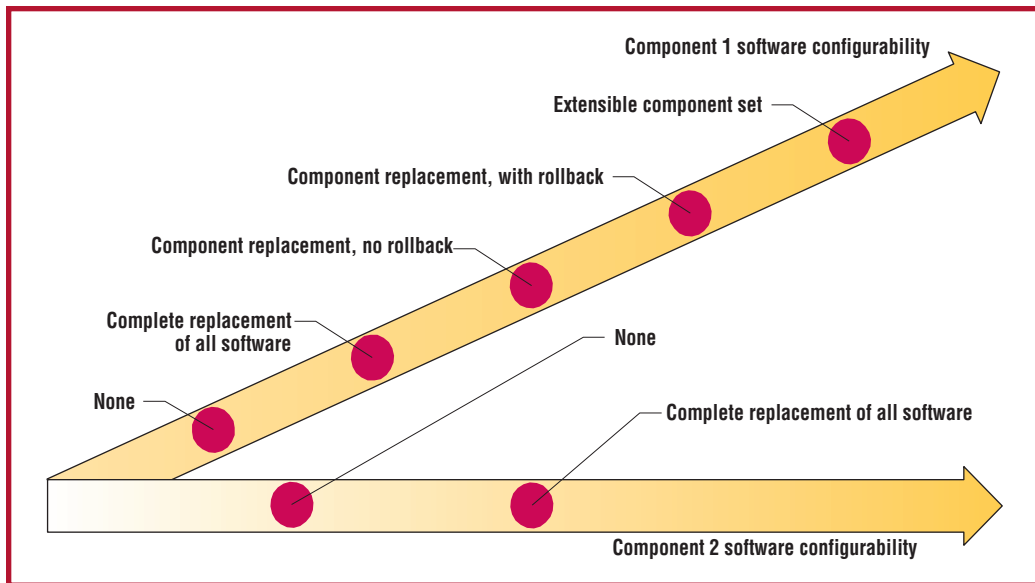


Figure 1. Two case study feature vectors: feature levels of software configurability for Component 1 and Component 2. The axes show how the feature levels are different for each of the features.

tional system or simplified its installation.

A *feature level* is a clear functional level that a feature implementation can achieve. Together, a feature and its levels define a *feature vector*. Figure 1 illustrates these key concepts using a graphical representation of two feature vectors from the case study, showing the feature levels for each (we've omitted the actual component names for confidentiality).

In our case study, the feature levels for Component 1's remote software configurability feature were

1. None
2. Complete replacement of all software
3. Replacement of components without rollback to the previous configuration
4. Replacement of components with rollback
5. An extensible, configurable software infrastructure/architecture (see Figure 1)

For any feature vector, each feature level should represent a distinct business value to customers and compare with the functional

capabilities of competitors' products. In addition, the difference between two feature levels should translate to a meaningful engineering development increment and represent an increase in perceived business value.

A feature's levels are often a monotonically increasing sequence or a partial order, as in the Component 1 example. On the basis of intuition, the product manager orders the levels for increasing value and levels of complexity. Each feature in the feature vector space should have between two and six levels defined. In our case study, the average number of feature levels for both hardware and software features was between four and five. Increases in functionality between levels should be modest—that is, implementations taking on the order of four to 10 weeks. Table 1 describes the feature levels for the Component 1 software configurability feature vector. In practice, we use a single spreadsheet for all such features.

A *release increment* is a collection of {feature, level} pairs that define the contents of

Table 1

Feature description table: Component 1 software configurability feature vector

Level	Description	Related features
All levels	This feature defines the support offered for remote replacement of the software in Component 1.	
1. None	Component 1 has no capability for remote software change.	
2. Complete replacement of all software	The application software in the component can be replaced completely. This solution does not support rollback.	
3. Component replacement, no rollback	The application software is decomposed into distinct components, and each of these can be independently replaced. This solution does not support rollback to restore the component being replaced if replacement fails.	Operations tools and Feature Level 2 required
4. Component replacement with rollback	As above, but with rollback if replacement fails.	
5. Extensible component set	The set of distinct components in the application can be extended, with the additional components loaded and replaced remotely.	

Table 2**Increment definition using feature levels**

Feature	Increment 1	Increment 2	Increment 3	Increment 4	Increment 5
Component 1 configurability	None				
		Complete replacement of the software, no rollback			
		Complete replacement of the software, with rollback			
			Component replacement, no rollback		
				Component replacement, with rollback	
					Component interrogation for software version
Component 2 configurability	None				
			Complete replacement of the software, no rollback		

the product release. A sequence of release increments represents increasing levels of functionality over time. In addition, by identifying feature levels of increasing functionality, such a sequence enables engineering to design the product intelligently, anticipating the changes.⁶

The features and levels are the lexicon over which our framework defines release increments. Our case study product included two software configurability feature vectors, one for each of two separate components. Table 2 shows our definition of how increments refer to feature levels: The feature column lists the feature name. The feature levels are distributed across the increment columns where the feature level is first implemented.

A *release* of a *releasable product* is a release increment judged to be worthy of field release. A release should show feature level improvements in six to 18 feature vectors. Typically, a product and its evolution include many more feature vectors, but many of these are either not consistent with the release objectives or are known to be of very limited market value. So, limiting improvements to six to 18 feature vectors has not proven difficult in practice, and the restriction allows for more focused analysis on the release's core features.

Feature vectors are not necessarily independent. It is important to capture interdependencies between features and their levels to ensure consistent decisions. When two features represent direct trade-offs, sometimes the customer makes the trade-off, on the basis of value; in other cases the product development

team makes engineering and related trade-offs during product development. For example, component size versus cost is a classic trade-off. Identifying such feature trade-offs during the planning process lets the product manager solicit input on the desired coexistence of features and levels.

Assessing the market ROI of feature levels

The ROI is the business value to the company of a product development effort investment. Our approach is based on the principle that a product manager's key role is to maximize the company's ROI for the development group. This principle is different from the goal of maximizing business value to customers. To estimate ROI, the product manager must understand the business value and the effort involved in developing the features.

Assessing feature business value

In our approach, business value estimates for different features do not have to be absolute; they can be relative. To calculate ROI, we only need to know, for instance, that some feature is twice as valuable as another according to some units. Ultimately, we assess the business value of a particular feature level—not the entire feature. Feature level value assessment gives the product manager more control to maximize the business value received for a feature level's development effort.

Selecting candidate features and levels

The product manager determines the features and levels to include in each of a prod-

uct's release increments by assessing the relative value of features and levels. The initial decisions identify the features' relative values. A typical software product might have hundreds of features. The objective is to reduce the number of features for which the product manager must identify and evaluate feature levels. (Later, we describe several techniques for achieving this reduction.)

Using the features' relative values, the product manager then selects a candidate subset of features for closer investigation. At this stage, it is unwise to exclude a feature only because its complete implementation would be too costly—a reduced functional level for that feature could be valuable to some customers. In our case study, the product manager chose about half of the 35 features as primary features.

For each feature from the candidate set, the product manager identifies several candidate levels for release increments. The relative values of these feature levels guide the release definition. Practically, the choices of features and levels are often interdependent.

Constituencies and value

Several stakeholder constituencies participate in or influence value assessment. Current customers require continued product maintenance. They are also a constituency for enhancements that can be sold as add-on or complementary products (“upsell”). Prospective customers require certain product features, and selling to these prospects might require demonstration features that are of greater value during the sales cycle than in the product's day-to-day use. In addition, other key constituencies that could influence overall feature and level value ratings include competitors with product offerings; industry and financial analysts; and internal sales, marketing, engineering, professional services, and support staff. In our case study, a key group of investors setting goals for the business formed one key constituency, and potential customers formed another. The investors controlled funding for the product development.

The product manager identifies the value to the company of the constituencies' product requests. Remember, we define value here in terms of the company's business goals. This includes an understanding of what the customers value but is primarily focused on what is best for the company.

For example, the company's goals might include increased revenue from new sales, revenue at higher margin, increased market share in a particular market, higher customer satisfaction and customer referenceability, and a clear functional differentiation with competitive products.

Without understanding the company's goals, product management cannot know which constituencies are most important and what to ask them. For example, prospective and current customer constituencies might figure differently in achieving the company's business goals. If increased revenue from new sales is the goal, the current customer constituency with enhancement requests is unlikely to contribute much toward achieving the goal. However, the prospective customer constituency is very relevant to this goal in terms of their desired features, platforms, third-party interfaces, and so on. Ultimately, the product manager must weigh the importance of each constituency's input.

We define a feature level's value as the relative likelihood that the appropriate target constituencies will respond to that feature level in a way that contributes to the company's business goals. In our case study, there was a clear difference in value between two stakeholder groups. One group would have accepted a minimal software configurability feature level at an earlier date in order to validate the product concept, whereas another group demanded a higher level of configurability.

The value survey

We assess value by surveying constituencies. The product manager collects value data using survey techniques that include Web surveys, targeted article surveys, key customer and prospect interviews, industry analyst interviews, win-and-loss reports in competitive situations, and business process and value chain modeling. The surveys evaluate relative values of features and feature levels and trade-offs between feature levels of different vectors. In our case study, for example, the feature vectors included one for device management and another for a specific aspect of operations management. We were interested in the relationship between these vectors' feature levels and, for example, the trade-offs constituencies made between them.

Several guides are available for creating useful surveys.⁷ For our focus on identifying

The product manager identifies the value to the company of the constituencies' product requests.

**Maximizing
information
from a small
sample requires
carefully
selecting
the surveyed
population and
adhering to
survey design
principles.**

the respondent's relative value assignments, we have found several basic question formats useful:

- *Quartile technique.* Constituents rate and evenly distribute the relative values of features and feature levels on a value scale.
- *Minimum acceptable feature levels.* Respondent selects the minimum acceptable level from a presented list.
- *Trade-off value of feature levels.* Survey determines relative values for two feature levels by presenting increments differing only by those feature levels.
- *Preference rankings.* Constituents rank combinations of features, showing the points at which the trade-off value changes with increasing feature levels.

Collecting the data

The surveys must maximize the information yield from a small survey effort. Product management can rarely get accurate information from a large range of constituency members, except for an existing product with a large user base that can be polled through Web surveys. Maximizing information from a small sample requires carefully selecting the surveyed population and adhering to survey design principles.⁷ The company's business goals imply the relative importance of constituencies, which guides the choice of survey participants.

Analyzing survey results

The survey analysis converts the value preferences expressed by respondents into assessments of the value to the company of a feature or feature level. Every preference expressed in a response might not have the same value, because some constituencies carry more weight. To arrive at values relevant for the company, we begin by classifying the expressed value preferences. We then calculate their contribution to the company's goals.

First, we rank the features and levels. We recommend that the features and feature levels be split into four or five ranges of value. We assign each response (defining a feature and a level) to a range. (Confidentiality concerns prevent us from providing ranking examples from our case study here.) We then weight each feature and level value recorded by the importance of the constituency the response

represents. Although this involves a numerical calculation, the calculation is approximate. Finally, we divide the value to the company into four or five ranges and allocate the weighted feature and level values from the survey to the company value ranges to achieve a feature's or level's value to the company.

Determining cost

In general, product development management identifies the engineering effort for some defined feature and feature level by applying past experience along with more rigorous software-effort or cost estimation techniques, such as Cocomo and function point analysis. We have successfully used a modified Delphi technique, by which we ask each developer on a team to independently provide effort estimations for each individual feature level. We then bring the group together, compare responses, and facilitate a group consensus on the estimates. We measure the resulting effort estimate for feature levels in standard units such as the person-week.

For each feature, the scope of each cost estimation is the incremental cost of moving one level along the feature vector—the effort it takes to implement a level, given that the previous level has already been implemented. Making this estimate for each individual feature level gives the product manager greater control over the contents of product increments. Because each estimate is incremental, the estimate of what is necessary to implement two or three feature levels for a given feature will probably be inflated. This occurs because additional engineering might be necessary to refactor one level's solution to move on to the next level. (We manage the overestimate later in the process; it doesn't reduce the approach's value.)

Level estimates include the required refactoring. Although it is often possible to provide a simple implementation of a low level for a feature, design and implementation complexity often increase at the higher levels. As higher feature levels are implemented, the existing implementations might need rework. The existence of a clear statement of levels in a feature vector format reduces the refactoring work without violating the agile process's focus on minimizing the work required to achieve a level. Typically, a feature level has several possible minimal implementations; knowing later

Table 3**Feature level value rating table example**

Cost	Commitments	Feature values (the shading indicates the next features to consider)	
		High	Low
Low—one to two person-weeks		Component 1 software configurability: replace all	Operations tools: feature level 2
Medium—two to eight person-weeks		Operations tools: feature level 3	Component 1 software configurability: component replacement with rollback
Medium-high—eight to 20 person-weeks	Availability of critical components from at least two suppliers	Component 1 cost (maximum for minimal system) less than \$x	Component 1 software configurability: component replacement, no rollback
High—20 to 50 person-weeks			

implementation goals can help show which choice will reduce future refactoring.

Examining the ROI

To illustrate the use of the value and cost analysis, Table 3 is an example feature level value-rating table, showing costs and values for three feature vectors: Component 1 software configurability, operations tools, and a hardware feature that bears directly on Component 1's cost. Table 3's columns are arranged from highest to lowest value; rows are arranged in increasing cost. Each cell contains the features whose value corresponds to the column heading and whose effort corresponds to the row heading. Table 3 also includes another reality of software product development—commitments the company has made to specific customers. (To preserve confidentiality, the actual contents of the cells do not reflect the real values in the case study.)

The features and feature levels with the highest ROI to the company are those in the upper left-hand corner of the table, where the highest value and lowest cost intersect. The shading indicates the next features to consider—those in cells nearest to the upper left-hand corner.

Determining increments and scheduling

Together, a set of features and a level for each of those features define a release increment. Each release increment is a candidate for external release. We advocate defining between two and six release increments between each external product release, depending on the external-release calendar. At the end of each increment's internal development and prior to any decision being made about which increment will become

the product release, it is important to reexamine the definitions of subsequent increments. Increments ensure that you have a release candidate at any time, provided you have completed the first increment.

Time is the principal determining factor in how much goes into each release increment. Some Extreme Programming proponents recommend two-week release increment intervals.³ Richard Selby and Michael Cusumano report that Microsoft uses a target of six to eight weeks.⁵ The product's deployment situation can also affect the choice of increment interval. For example, ASP applications' rapid, regular release model allows short increment intervals.

The increment content itself, selected to be consistent with the release objectives, depends on several factors:

- Engineering ROI, shown using a table such as Table 3
- Development resource constraints imposed by any required staff specializations
- The need for a releasable product with mutually consistent features and feature levels

Other factors can also determine the increment size and contents. For example, increments also allow the development of an initial version of a product, which can reassure investors and customers funding part of the development. Early increments can also serve as sales tools.

Engineering must review release increment definitions, as the increment might require less effort than the sum of the feature level implementation costs. This is because of the overestimation that results from the estimates referring to single feature levels in isolation.

About the Authors



Brian A. Nejme is the founder of Instep (www.instep.com), a product strategy firm, and a faculty member in the School of Mathematics, Engineering, and Business at Messiah College. Instep's offices are located in northern Virginia and suburban Philadelphia. His research interests include software product strategy, product planning, process modeling, and software engineering. He holds a BS from Allegheny College and an MS from Purdue University, both in computer science. Contact him at nejme@instep.com.

Ian Thomas is a consultant specializing in product planning and product management and their relationships with engineering processes and architectures. He has published in the fields of development environments and software processes. He has a BS from the University of Wales and an MS from London University. Contact him at ithomas@ix.netcom.com.

The short-term increment definitions are more detailed than the definitions for the longer-term increments; this is because the content of future release increments is subject to revision as each release increment is completed. Combined, the increment definitions constructed for several increments form the product development schedule.

We have applied this incremental planning approach in a variety of commercial settings, ranging from start-up technology ventures to long-lived product lines. Several key lessons emerged during this experience.

Defining product increments is only of value if you preserve your ability to release the product at any increment level, including an increment level achieved early in the development process. Version management tools and practices are critical for this purpose.

Once a release increment is selected as the release candidate, tasks involving integration, final documentation, additional testing, rollout preparedness, and release readiness (alpha, beta, and so on) must be completed. We advocate performing these tasks only on the release candidate—not on every single increment.

A company's product manager controls the flow of information to personnel in externally facing company roles, especially to those dealing with prospective and current customers. Thus, product management must control external expectations of the product and its release date. It is important to communicate throughout the company—especially with externally facing personnel—the incremental planning approach in use, including what information can and cannot be made public until authorized by the product manager.

Our experience also suggests directions for important future work. We are investigating the development of tools to support

and guide people through the underlying product-planning framework. Clearly such a product-planning toolset would have to be well integrated with other complementary software development tools, including project management tools. Second, we are exploring improvements to the techniques we have developed for feature level cost and value analyses, as the accuracy of these analyses affects the results of our approach. As we expand our use of the technique, we will incorporate additional principles for performing these analyses. In addition, points of integration between our product-planning framework and the product development process bear further exploration.

Finally, we are exploring ways to more effectively leverage the work of the conjoint analysis field in evaluating feature and feature level values.⁸ Conjoint analysis is currently used to analyze preferences for consumer goods. For example, the technique can illustrate the threshold where a customer will cease to pay additional amounts for increases in performance. We have used this technique, and it yielded useful information on trade-offs between levels for different features. 

Acknowledgments

We thank the *IEEE Software* reviewers and editors and Kevin Hickey for their many helpful comments.

References

1. A.K. Gupta and D.L. Wilemon, "Accelerating the Development of Technology-Based New Products," *Calif. Management Rev.*, vol. 32, no. 2, Winter 1990, pp. 24–44.
2. E. Raymond, *The Cathedral and the Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary*, O'Reilly Press, Sebastopol, Calif., 1999.
3. M. Fowler and J. Highsmith, "The Agile Manifesto," *Software Development*, vol. 9, no. 8, Aug. 2001, pp. 28–32.
4. K. Wiegers, "First Things First: Prioritizing Requirements," *Software Development*, vol. 7, no. 9, Sept. 1999, pp. 48–53.
5. M. Cusumano and R. Selby, *Microsoft Secrets: How the World's Most Powerful Software Company Creates Technology, Shapes Markets and Manages People*, Free Press, New York, 1995.
6. D. Parnas and P. Clements, "A Rational Design Process: How and Why to Fake It," *IEEE Trans. Software Eng.*, vol. SE-12, no. 2, Feb. 1986, pp. 251–257.
7. F. Fowler, *Improving Survey Questions: Design and Evaluation*, Sage Publications, Thousand Oaks, Calif., 1995.
8. A. Gustafsson, A. Herrmann, and F. Hubers, eds., *Conjoint Measurement: Methods and Applications*, 2nd ed., Springer-Verlag, Berlin, 2001.

For more information on this or any other computing topic, please visit our Digital Library at <http://computer.org/publications/dlib>.